

DPI343

Computer Organization and Assembly Language

Lecture 14

Keyboard

Lectures - prof. em. U. Sukovskis

Labs for local students - assoc. prof. P. Rusakovs

Labs for foreign students - assoc. prof. G. Alksnis

Autumn 2025



RTU
DATORZINĀTNES,
INFORMĀCIJAS
TEHNOLOĢIJAS
UN ENERĢĒTIKAS
FAKULTĀTE

PC keyboard



- The original IBM PC keyboard **generates** the key down **scan code** when you press a key and the key up scan code when you release it
- The keyboard microcontroller transmits these **scan codes to the I/O port 60h** and **the hardware interrupt 9** is generated and processed by the interrupt handler (for example, BIOS keyboard interrupt handler).
- Examples of scan codes: Esc - 1, !/1 - 2, @/2 - 3, ...
- Some keys (cursor keys, Insert, Del, PgUp, etc.) generate byte sequences (→ - E0 4D, Del - E0 53, PgDn - E0 51, PrnScr - E0 2A E0 37, etc.)

Keyboard scan codes (hex)

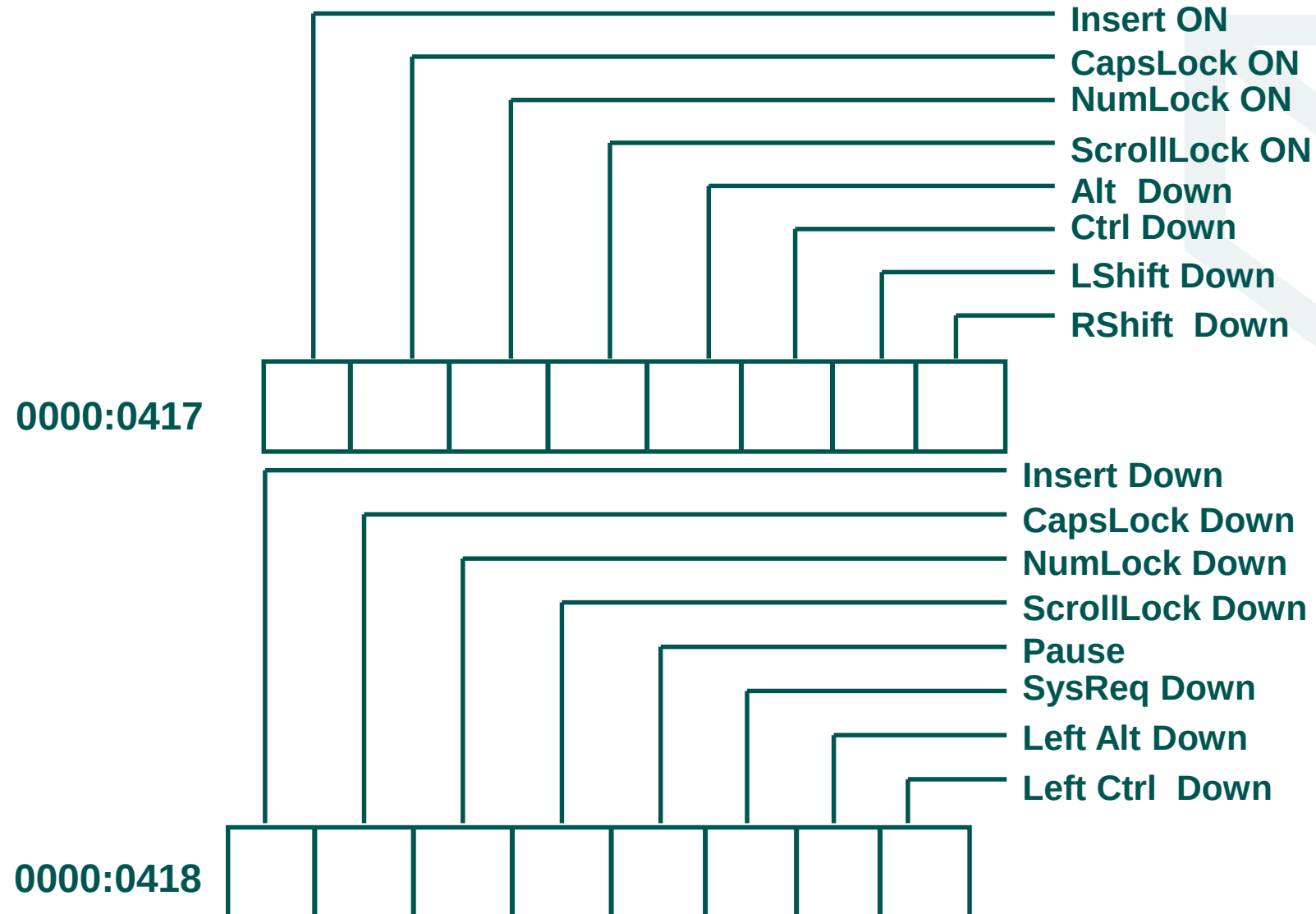
Key	Down	Up	Key	Down	Up	Key	Down	Up	Key	Down	Up
Esc	1	81	[{	1A	9A	, <	33	B3	<i>center</i>	4C	CC
1 !	2	82	] }	1B	9B	. >	34	B4	<i>right</i>	4D	CD
2 @	3	83	Enter	1C	9C	/ ?	35	B5	+	4E	CE
3 #	4	84	Ctrl	1D	9D	R shift	36	B6	<i>end</i>	4F	CF
4 \$	5	85	A	1E	9E	* PrtSc	37	B7	<i>down</i>	50	D0
5 %	6	86	S	1F	9F	alt	38	B8	<i>pgdn</i>	51	D1
6 ^	7	87	D	20	A0	space	39	B9	<i>ins</i>	52	D2
7 &	8	88	F	21	A1	CAPS	3A	BA	<i>del</i>	53	D3
8 *	9	89	G	22	A2	F1	3B	BB	/	E0 35	B5
9 (	0A	8A	H	23	A3	F2	3C	BC	<i>enter</i>	E0 1C	9C
0)	0B	8B	J	24	A4	F3	3D	BD	F11	57	D7
- _	0C	8C	K	25	A5	F4	3E	BE	F12	58	D8
= +	0D	8D	L	26	A6	F5	3F	BF	ins	E0 52	D2
Bksp	0E	8E	; :	27	A7	F6	40	C0	del	E0 53	D3
Tab	0F	8F	‘ “	28	A8	F7	41	C1	home	E0 47	C7
Q	10	90	~	29	A9	F8	42	C2	end	E0 4F	CF
W	11	91	L shift	2A	AA	F9	43	C3	pgup	E0 49	C9
E	12	92	\	2B	AB	F10	44	C4	pgdn	E0 51	D1
R	13	93	Z	2C	AC	NUM	45	C5	left	E0 4B	CB
T	14	94	X	2D	AD	SCRL	46	C6	right	E0 4D	CD
Y	15	95	C	2E	AE	<i>home</i>	47	C7	up	E0 48	C8
U	16	96	V	2F	AF	<i>up</i>	48	C8	down	E0 50	D0
I	17	97	B	30	B0	<i>pgup</i>	49	C9	R alt	E0 38	B8
O	18	98	N	31	B1	-	4A	CA	R ctrl	E0 1D	9D
P	19	99	M	32	B2	<i>left</i>	4B	CB	Pause	E1 1D 45 E1 9D C5	-

Keyboard interrupt processing by BIOS

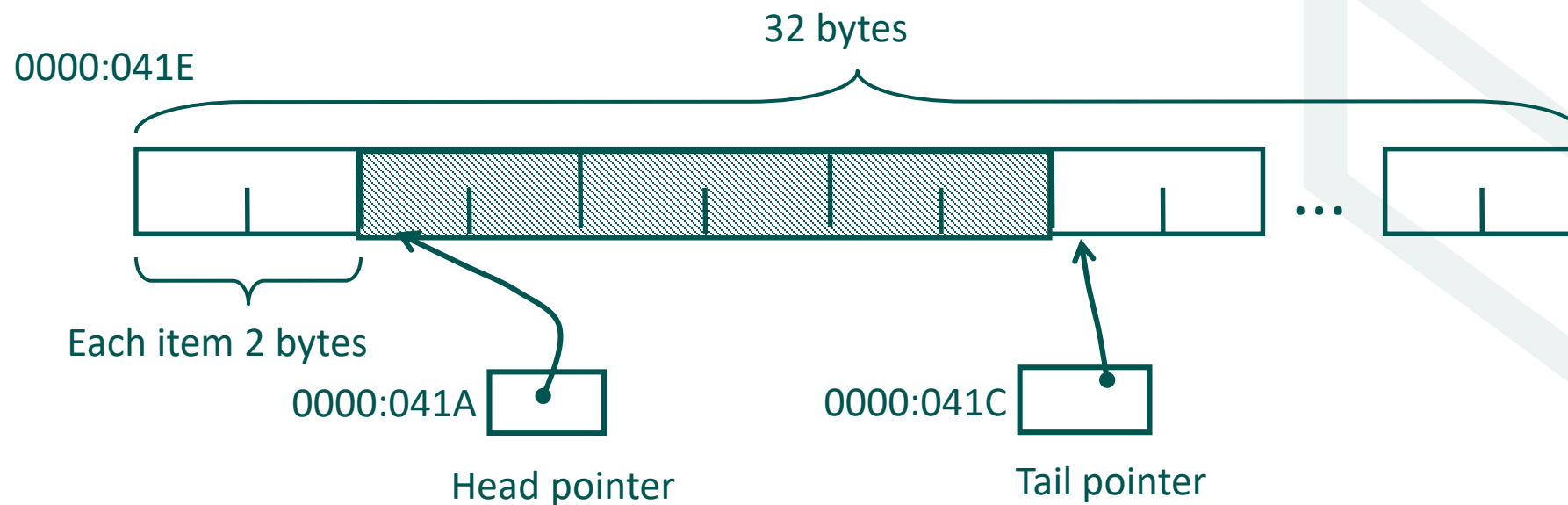
Standard BIOS keyboard interrupt handler:

- Reads scan code from the **port 60h**
- If the interrupt was caused by a character key then corresponding ASCII code is loaded into the **keyboard buffer**
- If Alt or Ctrl key is held down while pressing a character key or a function key (F1-F12) is pressed, then so called **extended code** is loaded (Alt-A - 30, Alt-B - 48, F1 – 59, etc.)
- If the interrupt was caused by a **status key** (Shift, Ctrl, Alt, NumLock, etc.) then **keyboard status bits (shift flags) are set** (BIOS Data Area bytes at 0000:0417 and 0000:0418)

Keyboard status bits



Keyboard Buffer

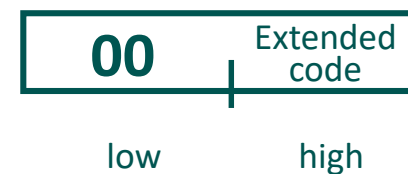


If value of the *Head pointer* is equal to *Tail pointer*, the buffer is empty.

Each two bytes of the buffer item contains



or



Example

Write a code to show message “Ctrl key is down” if Ctrl key is held down at the moment the code is executed.

1. Where you can find information about Ctrl key status?

Byte at 0000:0417, bit number 2

2. How to copy memory byte from 0000:0417 into the register AL?

```
xor    bx, bx
mov    es, bx
mov    al, es:0417h
```

or

use BIOS service interrupt INT 16h to read status flags

```
mov ah, 12h    ;BIOS Service - Get Shift States
int 16h        ;returns status bits in AL and AH
```

3. How to check the value of one bit in the register AL?

```
test    al, 00000100b
jz      noctrl
```

4. How to show a message?

```

;...
.data
msgInit byte      "Press any key or Esc to exit!", 13, 10, "$"
msgCtrl byte      "Ctrl key is down", 13, 10, "$"
.code
main proc
.startup
; Show initial message
    mov ah, 9
    mov dx, offset msgInit
    int 21h
; Check status bit of the Ctrl key at 0000:0417
check:  xor bx, bx
        mov es, bx
        mov al, es:0417h
        test al, 00000100b
        jz  noctrl
; Show message that Ctrl key is down
    mov ah, 9
    mov dx, offset msgCtrl
    int 21h
; Wait for key press
noctrl: mov ah, 0          ; function 0 - wait for key press
        int 16h           ; BIOS service for keyboard (returns: ah=scan code, al=ASCII)
; Check scan code of the last key pressed
        cmp ah, 1         ; Esc?
        jne check
.exit
main endp
end

```


How to write our own keyboard interrupt handler?

What we already know

- Keyboard interrupt is a hardware interrupt
- Keyboard controller uses IRQ1 of PIC (Programmable Interrupt Controller)
- So the number of the keyboard interrupt is 9 ($1 + 8 = 9$)
- Entry point of the keyboard interrupt service routine is in the interrupt vector No. 9 (four memory bytes at address 0000:0024)
 $9 * 4 = 36 \text{ dec} = 24 \text{ hex}$
- We are going to write an Assembly procedure which will receive control each time when keyboard interrupt happens and store the entry point address of this procedure in the interrupt vector No. 9
- We'll use a program structure very similar to the one we used for timer interrupt handling

Example

Task

If both **Ctrl** and **LeftShift** keys are held down and **Backspace** key is pressed then indicate this combination of keys by writing letters «KB» in the middle of the text screen

- Our own interrupt handler will be executed on each keystroke
- The interrupt handler will check status of keys (Ctrl, LeftShift) and scan code of the last key pressed
- If the combination of keys is as required, then it shows letters «KB» on the screen using direct writing to video memory
- In the case the combination of keys is not one we are interested in, control will be transferred to the standard (previous) interrupt handler (it could possibly be the BIOS code)
- Our handler will replace the standard keyboard interrupt handler, but it should know the address of the entry point to the standard handler to be able to transfer control to it

Structure of the TSR source code

```
.model small
```

```
.code
```

```
int9h    proc far    ; interrupt 9 handler
          push es    ; save registers
          push ax
          :::        ; Check combination of keys and perform activities
          pop  ax
          pop  es
          iret

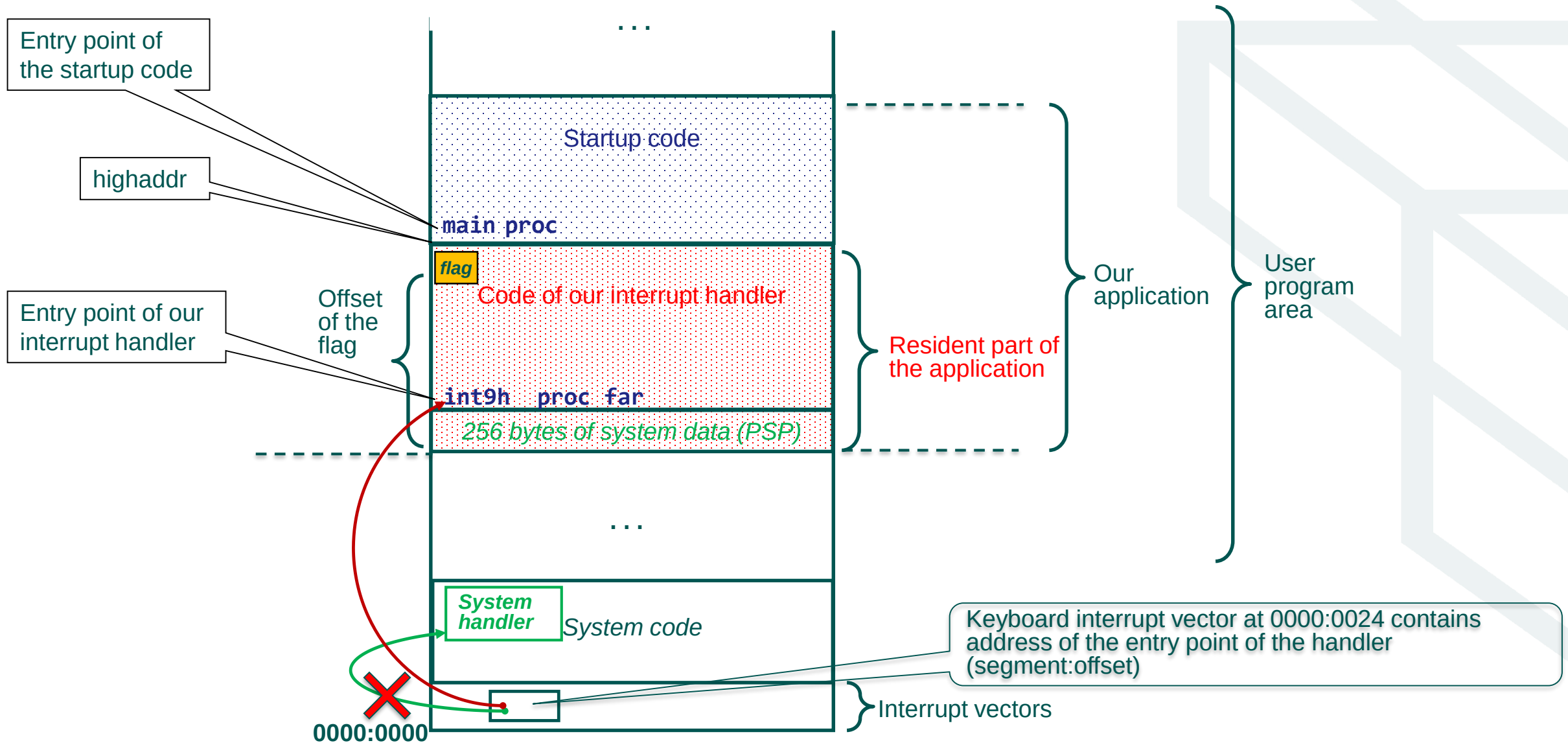
int9h    endp
; Data used by the interrupt handler
:::
highaddr:
```

This part of the code remains in memory

```
main     proc        ; Entry point of the startup code
          :::
;-----Get and save entry point address of the current handler
          :::
;-----Set new interrupt vector
          :::
;-----Show message on the screen that new handler is active
          :::
;-----Exit and keep the handler in memory
          :::
main     endp
end main
```

This part of the code is executed on startup

What does it look like in the memory?



```

;-----Keyboard interrupt handler procedure
int9h  proc    far
    push  es    ; save registers
    push  ax
    push  bx
    push  di
    sti           ; set interrupt flag
;-----Check the current combination of the keyboard keys
    mov  bx, 0
    mov  es, bx    ; segment register es = 0
    mov  ah, es:0417h
    and  ah, 00000110b ; Ctrl and LShift are down?
    cmp  ah, 00000110b
    jne  retold    ; if not, hand over control to the previous ISR
    in   al, 60h    ; read scan code of the last key pressed
    cmp  al, 14    ; BackSpace scan code ?
    jne  retold    ; if not, hand over control to the previous ISR
;-----Perform activities if the combination currently is on
    mov  bx, 0B800h
    mov  es, bx
    mov  di, 1996    ; offset in the video memory
    mov  byte ptr es:[di], 'K'
    mov  byte ptr es:[di+1], 01110100b ; red letter on white background
    mov  byte ptr es:[di+2], 'B'
    mov  byte ptr es:[di+3], 01110010b ; green letter on white background
;-----Return from hardware interrupt handler

```

```

;-----Return from hardware interrupt handler
    mov al, 20h      ; inform PIC that processing is done (EOI)
    out 20h, al
    pop di           ; restore registers
    pop bx
    pop ax
    pop es
    iret

;-----Transfer control to the previous interrupt handler
retold:
    pop di           ; restore registers
    pop bx
    pop ax
    pop es
    jmp cs:[oldint9] ; jump to entry point of the previous handler

int9h    endp

;-----Data used by the interrupt handler
oldint9    dword    0
flag       byte     'LRKBDU'

highaddr:      ; Keep the code in memory up to this address

```

```

;=====Startup code=====
main      proc          ; Entry point of the startup code
          pushcs        ; Data is stored in the code segment
          pop ds         ; so copy cs to ds
;----Prevent duplicate copies of the same interrupt handler in memory
          ; . . . some code to check the flag
activate:
;-----Get and save entry point address of the current (previous) interrupt ISR
          mov ax, 3509h   ; function 35h interrupt 09h
          int 21h         ; read interrupt vector in es:bx
          mov word ptr oldint9, bx ; save offset of the entry point address of the current ISR
          mov word ptr oldint9+2, es ; save segment of the entry point address of the current ISR
;-----Set new interrupt vector - entry point address of our ISR int9h
          mov dx, offset int9h ; dx = entry point offset of our ISR, ds = cs
          mov ax, 2509h       ; function 25h, interrupt 09h
          int 21h             ; write interrupt vector from ds:dx
;-----Show message on screen that new ISR is active
          lea dx, msgOK
          mov ah, 9
          int 21h
;-----Exit and keep the ISR code in memory
          mov dx, offset highaddr
          add dx, 10Fh        ; add size of the PSP and round up to the paragraph
          shr dx, 4           ; divide by 16
          mov ax, 3100h       ; function 31h, return code 00h
          int 21h             ; return to operating system and keep the code of our ISR in memory
main      endp

```